

Module 1 Foundations and the Python Bridge

Overview and study route

Software Engineering in the Era of AI | Master's year one | Electrical and Information Engineering | Weeks 1 and 2 | 2026 to 2027

Software engineering turns a need into software whose behaviour can be explained, checked and maintained. AI changes how quickly candidate solutions can be produced, but it does not decide what counts as a correct engineering result. A plausible program can use the wrong physical model, mix units, ignore missing measurements or return an impressive explanation without evidence.

This module establishes a common starting point for students with limited programming experience. The core task is to calculate energy from a small sequence of power readings. You will move from an engineering statement to a specification, a hand calculation, a Python function and a set of checks. The master's-level question is how the assumptions and evidence limit the conclusion, even when the code runs correctly.

Read Sections 1 to 5 for Week 1 and Sections 6 to 10 for Week 2. Work through each trace before running the supplied examples. Core work is required; extension questions deepen the reasoning without making advanced syntax a prerequisite. Budget four to six hours of independent study per week alongside the scheduled contact time.

Learning outcomes

- Explain requirements, design, implementation, verification, validation and maintenance using one engineering example.
- Distinguish AI-assisted development from software that contains an AI component.
- State input domains, units, assumptions and observable acceptance criteria.
- Read and modify Python expressions, conditions, loops, lists and functions.
- Trace a short program and compare its result with an independently calculated expectation.
- Explain an invariant, a boundary case and a simple linear-time algorithm.
- Record permitted AI assistance while retaining responsibility for the result.

These outcomes support course LO1, LO2 and LO3. No previous Python knowledge is required for the core route.

1 Software engineering as a chain of evidence

A requirement describes needed behaviour or a constraint. A design chooses a structure capable of meeting it. An implementation expresses that design in executable form. Verification checks whether an artifact satisfies a specification; validation asks whether the specification and resulting system address the actual need. Maintenance changes software as requirements, defects, environments and dependencies change.

Suppose a laboratory manager wants an equipment energy report. A program might correctly sum every reading but treat samples taken one minute apart as samples taken one hour apart. Its arithmetic can be

internally consistent while its interpretation is wrong. Verification must include the interval contract; validation must establish that the chosen sampling model is suitable for the manager's decision.

The lifecycle is iterative. A failed test may reveal an implementation error. A misleading result may reveal an inadequate requirement. A deployment failure may reveal an undocumented environment assumption. Keep a trace from the need to the evidence so that you know what to revise.

Artifact	Example	Evidence it should enable
Requirement	Report energy in Wh from interval-average power	A reader can determine what a correct output means
Design	Separate validation, calculation and reporting	Each responsibility can be changed and checked
Implementation	A Python energy function	The design can execute on supplied data
Test	Two 60 W intervals of 30 minutes yield 60 Wh	A concrete requirement is checked
Operational record	Input dataset and software release identifier	A result can be reproduced and investigated

Abstraction exposes what a component does without requiring every internal detail. Modularity assigns related responsibilities to components with explicit boundaries. Reuse is useful when a component's assumptions match a new context; copying code whose units or error policy differ is not safe reuse. Maintainability depends on understandable contracts and change boundaries, not merely short source files.

2 Two different uses of AI

In AI-assisted development, an assistant helps a person write, explain, review or change ordinary software. The final energy function can be deterministic even if AI helped produce it. In an AI-enabled application, a model is invoked while the application runs, for example to explain a maintenance paragraph. That runtime component may produce variable output and requires separate evaluation and failure handling.

An autocomplete suggestion, a chat answer and a coding agent have different authority. A suggestion cannot change a repository unless someone accepts it. An agent may read files, edit code, execute commands and make additional decisions. Current agent documentation illustrates this expanded scope. [R1] More authority makes task boundaries and inspection of actual changes more important.

Use AI to propose alternatives and expose assumptions, then evaluate the proposal. A confident explanation is not an acceptance test. When a tool says that it ran tests, inspect the recorded result and understand what was tested. A generated implementation and generated tests may share the same misunderstanding.

For this course, use **Independent** mode for initial predictions and concept checks; **Guided** mode for explanations after an attempt; and **Bounded development** mode for a specified implementation task. An assistance record states the date, tool, task, accepted changes and independent checks. It need not contain hidden model reasoning or every conversational turn.

3 Specify the engineering model before coding

Energy is the integral of power over time. For interval-average power values P_i in watts and equal interval duration d in minutes, our discrete model is $E_{Wh} = \sum(P_i) \times d / 60$. Each value represents one complete interval. Three values describe three intervals, not three instantaneous samples defining two intervals.

This distinction matters. If measurements are instantaneous samples at timestamps, integration may instead require the differences between timestamps and an interpolation assumption. The trapezoidal rule can be appropriate for that different contract. Neither formula should be selected merely because an AI tool recognises the word energy.

Core contract. Accept a nonempty list of built-in Python integers or floats representing finite, non-negative interval-average power. Accept a finite positive interval in minutes. Reject booleans, non-numeric entries, missing values, negative values and non-finite values. Return a finite float in Wh. Reject a calculation outside the finite numeric range. A physically zero-power interval is valid. Signed net power for export is outside this teaching contract.

The restriction to non-negative power is a modelling choice, not a universal law. A later net-metering service could accept negative export power, but its name, tests and reports would need a revised contract. A missing sample must not silently become zero because unknown consumption and zero consumption are different observations.

Worked example

Take interval-average readings [60, 120, 0] W, each lasting 30 minutes. Each interval is 0.5 hours. The contributions are 30 Wh, 60 Wh and 0 Wh, giving 90 Wh. In kilowatt-hours this is 0.09 kWh. The energy is not 90 kWh.

The same total power sum with 15-minute intervals gives 45 Wh. Doubling every power reading doubles energy if the duration and all other assumptions remain unchanged. These relationships give independent checks beyond one numerical example.

Case	Data	Expected behaviour
Ordinary	[60, 120, 0], 30 minutes	90 Wh
Zero power	[0], 10 minutes	0 Wh
Empty data	[], 30 minutes	Reject
Invalid interval	[60], 0 minutes	Reject
Unknown reading	[60, None], 30 minutes	Reject
Non-finite reading	[infinity], 30 minutes	Reject

4 Start with a script and a trace

An editor saves source text. An interpreter executes the program. A script is a saved file ending in .py; its output is not the source file. Running the wrong saved copy is a common source of confusing results. Begin with a known course folder and give each example a descriptive filename.

```
powers_W = [60.0, 120.0, 0.0]
interval_min = 30.0
energy_Wh = sum(powers_W) * interval_min / 60.0
print("Energy (Wh):", energy_Wh)
```

This fixed-data demonstration prints Energy (Wh): 90.0. Square brackets create a list. An equals sign binds a name to a value. The function sum adds the entries. Parentheses after print supply its arguments. Quoted text is displayed literally, while an unquoted expression is evaluated.

The suffixes W, min and Wh help the reader; Python does not infer physical units from them. A variable named interval_min can still contain a value measured in seconds. Meaning comes from the contract and input handling.

Predict what changes if interval_min becomes 15.0. The answer is 45.0 Wh. Changing only the printed label from Wh to kWh does not convert the numerical value. A unit conversion requires division by 1000 as well as an appropriate label.

Environment steps

Open the approved editor, create a plain text file named first_energy.py and enter the script. Save it before execution. Use the editor's Python run command or run python first_energy.py in a terminal opened in that folder. If your installation uses py or python3, use that approved interpreter command consistently. Do not paste an interactive prompt such as >>> into the file.

Record the interpreter version, the expected output and the actual output. Reopen the saved file to confirm that the evidence points to the executed source. The core examples need no third-party package and no AI service account.

5 Values and expressions

Python integers represent whole numbers; floats represent finite-precision approximations to many real numbers; strings represent text; booleans represent True or False. A value such as "60" is a string until explicitly converted. Dividing by 60 converts minutes to hours in this model. The operator / performs true division, whereas // performs floor division and would lose the fractional interval.

Boolean comparisons produce values used in decisions. $x < 0$ is true for a negative number. $x == 0$ tests equality; $x = 0$ assigns a value. Confusing the two is both a syntax and a reasoning risk depending on context. Use parentheses when the intended grouping might be unclear.

Lists hold ordered collections and use zero-based indexing. powers_W[0] selects the first entry. A dictionary maps keys to values, such as {"device": "motor_1", "power_W": 60.0}. Its key names form a data contract with other parts of the program. A missing key is different from a present key whose value is None.

Python's official tutorial documents expressions, control flow and data structures. [R2] Use it to clarify language behaviour, then return to the engineering meaning of the example.

6 Decisions loops and state

An if statement chooses a block according to a condition. Indentation groups the statements belonging to that block. A for loop repeats a block for each item in a collection. A running total is state: its value changes as the loop progresses.

```
total_W = 0.0
for power_W in [60.0, 120.0, 0.0]:
    if power_W < 0:
        raise ValueError("negative power is outside this model")
    total_W = total_W + power_W
print(total_W)
```

This code prints 180.0. It shows one domain check, not the full input validation contract. A string entry would cause a type error; a non-finite entry could slip through this particular check. State precisely what a demonstration implements.

Completed iterations	Latest reading	Running total
0	None processed	0 W
1	60 W	60 W
2	120 W	180 W
3	0 W	180 W

The loop invariant is that after k iterations, `total_W` equals the sum of the first k accepted readings. Initially it holds because no readings have been processed and the total is zero. Processing the next reading preserves it. At termination it gives the sum of all readings. This argument explains the algorithm; it does not by itself prove the correctness of input validation or floating-point arithmetic.

7 Functions and explicit contracts

A function gives a reusable computation a name, parameters and a return value. Parameters are local names for the inputs to one call. A return statement passes a result back to the caller. Printing a value and returning a value are different: a later calculation needs the returned value.

The supplied `energy.py` contains the complete reference implementation. Read it in layers: first the signature, then the input checks, then the calculation. Type annotations describe intended use and assist tools; they do not automatically validate runtime inputs. A docstring states the model and error behaviour for a human reader.

```
def interval_energy_Wh(powers_W, interval_min):
    """Small core example; inputs are assumed already validated."""
    total_W = 0.0
    for power_W in powers_W:
        total_W += power_W
    return total_W * interval_min / 60.0

print(interval_energy_Wh([60.0, 120.0, 0.0], 30.0))
```

The shorter example isolates function mechanics. Its precondition is already-validated data. Do not advertise it as a general input-validation solution. The complete supplied version rejects boolean values because Python treats `bool` as a subclass of `int`, yet `True W` is not a valid measurement under our contract.

8 Errors and debugging

A syntax error prevents source from being interpreted correctly, for example a missing closing parenthesis. An exception during execution signals a problem such as an invalid conversion or rejected domain value. A logic error can produce the wrong answer without raising an exception.

Debug by recording the smallest input that reproduces the problem, the expected result and the observed result. Read the traceback from its exception message and locate the relevant source line. Inspect intermediate values against the model. Change one cause at a time and rerun both the failing case and a previously correct case.

Suppose a function returns $\text{sum}(\text{powers_W}) / \text{len}(\text{powers_W})$. It computes mean power, not energy. It may produce 60 for [60, 120, 0], but that value has units W. Adding a Wh label does not repair the algorithm. A test with 30-minute intervals should expose the mismatch because the required result is 90 Wh.

Avoid hiding an exception with a broad catch that returns zero. The report would then convert a failed calculation into apparently valid evidence. A useful failure message should distinguish invalid data from a successful zero result.

9 Complexity numerical limits and abstraction

Summing n readings examines each entry once, so its running time is $O(n)$. For the loop's additional state, memory is $O(1)$, excluding the already-stored input list. An implementation that first creates a validated copy also uses $O(n)$ additional memory. Always state what is included in a space claim.

Floating-point results are approximations. Do not demand exact equality for every computed decimal. Later tests will use an absolute or relative tolerance justified by the measurement context. Sensor uncertainty, sampling assumptions and numeric roundoff are separate sources of error; improving numeric precision cannot recover an unmeasured interval.

Extension. If each interval-average power has an absolute error bounded by e W and there are n equal intervals of d minutes, a conservative absolute energy error bound is $n \times e \times d / 60$ Wh. This assumes the duration is exact and does not model correlated calibration bias separately. For ten half-hour intervals with $e = 2$ W, the bound is 10 Wh. A small software roundoff may be negligible beside that measurement bound.

10 Common mistakes and research habits

Do not equate successful execution with valid engineering. Do not replace missing readings with zero without a documented policy. Do not compare software versions on different input datasets and call the difference an AI effect. Do not infer that a tool is always faster from one enjoyable interaction.

A useful technical claim states its scope: which task, data, version, outcome and uncertainty. For example, "the supplied local function produced 90 Wh on these three interval-average readings" is supported by a specific run and manual calculation. "AI makes engineering reliable" is not established by that evidence.

Weekly laboratories

Week 1 lab From model to executed script

Preparation: have the approved interpreter and editor ready. Use Independent mode for predictions and Guided mode for explanation afterwards.

- Minutes 0 to 10: specify the units, input meaning and expected result for [60, 120, 0] over 30-minute intervals.
- Minutes 10 to 25: save and execute `first_energy.py`; record actual output and source location.
- Minutes 25 to 40: change the interval to 15 minutes, predict first, then run; convert the answer to kWh.
- Minutes 40 to 50: critique a supplied candidate that divides by the number of readings instead of converting time.
- Minutes 50 to 60: explain one modelling assumption and submit the script with its prediction and observation table.

Completion evidence: a runnable script, two independently calculated results and a short explanation of the candidate's error. An account screenshot without source is insufficient.

Week 2 lab A reusable validated computation

Preparation: read the supplied `energy.py` and write a trace for the loop. Work in pairs if useful, with each student explaining one independent case.

- Minutes 0 to 10: identify the function's inputs, output and rejection policy.
- Minutes 10 to 30: run ordinary, zero-power and invalid cases; inspect how an exception differs from a zero result.
- Minutes 30 to 45: modify a copy to report kWh through a separate wrapper, preserving the Wh function.
- Minutes 45 to 55: explain why a negative reading and an empty list are rejected.
- Minutes 55 to 60: record what you changed and which independent checks support it.

Extension after class: revise the specification for signed net power and describe the tests that would need changing before altering the code.

Practice questions

Question 1 Specification

Write a contract for energy from [100, 200] W interval averages with a 15-minute interval. Include units, input meaning and the expected answer.

Model answer. Each value represents a full 15-minute interval. Inputs must satisfy the finite non-negative contract. Energy is $(100 + 200) \times 15 / 60 = 75$ Wh, or 0.075 kWh. State that instantaneous samples would require a different integration assumption.

Question 2 Verification and validation

A correctly implemented service reports hourly energy, but the operator needs peak instantaneous current. Has the software necessarily solved the user's problem?

Model answer. No. It may satisfy its energy specification, but that specification does not answer the peak-current need. Validation must revisit the required quantity, available data and physical model. More tests of the same energy formula cannot fix the mismatch.

Question 3 Output tracing

Predict the output of `print("60 / 2")` and `print(60 / 2)`.

Model answer. The first displays the text `60 / 2`. The second evaluates division and displays `30.0`. Quotation marks distinguish text from an arithmetic expression.

Question 4 Units

An AI candidate reports 500 kWh for a 1000 W device operating for 30 minutes. Evaluate it.

Model answer. The duration is 0.5 h, so energy is 500 Wh = 0.5 kWh. The candidate is too large by a factor of 1000 in the stated unit. Check the calculation and conversion, not just the plausibility of the prose.

Question 5 Missing data

Why is replacing None with 0.0 an engineering decision rather than a harmless data-cleaning step?

Model answer. None denotes an unknown observation. Zero asserts that no power was consumed during that interval. Substitution changes the model and can underestimate energy. A policy might reject, estimate with uncertainty, or explicitly report incomplete coverage, but it must be stated and tested.

Question 6 Loop invariant

State the running-total invariant and trace it for [10, 20, 30].

Model answer. After k iterations the total equals the sum of the first k readings. The totals are 0 before the loop, then 10, 30 and 60. Initialisation, preservation and termination connect the loop to the required sum.

Question 7 Complexity

Explain the time and additional-memory complexity of a loop that sums an existing list without copying it.

Model answer. Time is $O(n)$ because each entry is processed once. Additional memory is $O(1)$ for the accumulator and loop state, excluding the input list. A separate validated copy would change additional memory to $O(n)$.

Question 8 Domain boundaries

Classify `[]`, `[0]`, `[-1]`, `[True]` and `[float("nan")]` under the stated energy contract, using a 30-minute interval.

Model answer. Only `[0]` is valid and returns 0 Wh. Empty input, negative power, boolean pseudo-measurements and non-finite values are rejected. Arithmetic permissibility is not the same as membership in the chosen domain.

Question 9 Function behaviour

Why should the core energy function return a number rather than only print it?

Model answer. A returned number can be tested, converted, stored or passed to another computation. Printing is a presentation side effect. Separating the two permits reuse without parsing display text.

Question 10 AI roles

An assistant writes a deterministic threshold function. A second application calls a language model to explain a maintenance note. What differs in their verification needs?

Model answer. The threshold function needs code review and tests against its specified deterministic behaviour. The runtime model feature additionally needs an evaluation of variable outputs, grounding, refusal or fallback behaviour and operational limits. AI authorship alone does not make the threshold calculation probabilistic at runtime.

Question 11 Error bound

For six 20-minute intervals with power uncertainty bounded by 3 W per interval, calculate the conservative energy error bound.

Model answer. The bound is $6 \times 3 \times 20 / 60 = 6$ Wh, assuming exact duration. This is a worst-case additive bound, not a probabilistic confidence interval or a claim that the actual error is 6 Wh.

Question 12 Independent evidence

An agent says its energy implementation is correct because all tests it wrote pass. Give two additional checks and explain their purpose.

Model answer. Use a manually derived case with non-hourly intervals to challenge the units and interpretation. Add an independently selected invalid case such as non-finite power to challenge the domain. Review the tests against the specification, since generated code and generated tests may share one wrong assumption.

Revision checklist

Explain the difference between the physical model and its implementation. Trace a function without running it. State one valid zero case and one invalid input. Calculate energy and its units manually. Distinguish time complexity from measurement error. Explain what an assistant changed and what evidence you checked independently.

Sources and further reading

- [R1] GitHub. Application card for GitHub Copilot Agents.
<https://docs.github.com/en/copilot/responsible-use/agents>
- [R2] Python Software Foundation. Python tutorial, especially introductions, control flow and data structures. <https://docs.python.org/3/tutorial/>
- Further reading: Sommerville, Software Engineering, chapters on software processes and requirements; Python documentation on floating-point arithmetic.
<https://docs.python.org/3/tutorial/floatingpoint.html>

Online references checked 15 September 2026. Numerical scenarios and questions are original classroom examples. The supplied scripts use simulated data and do not control physical equipment.